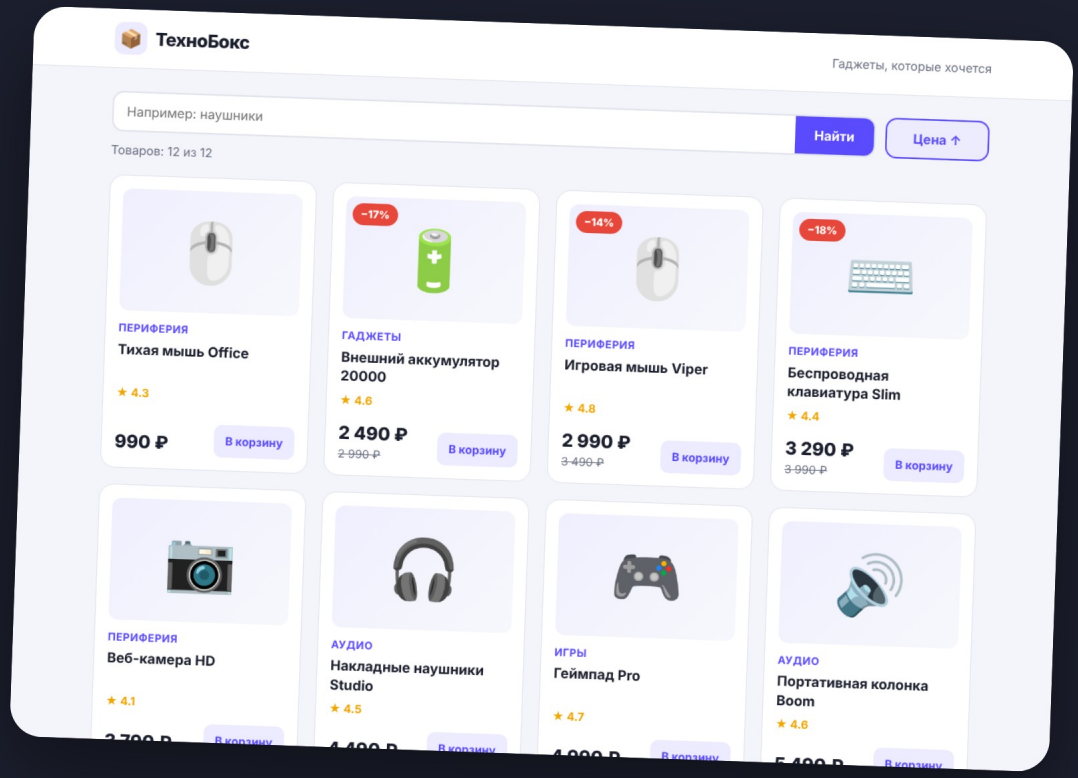
ПРАКТИЧЕСКИЙ ПРОЕКТ

ТехноБокс

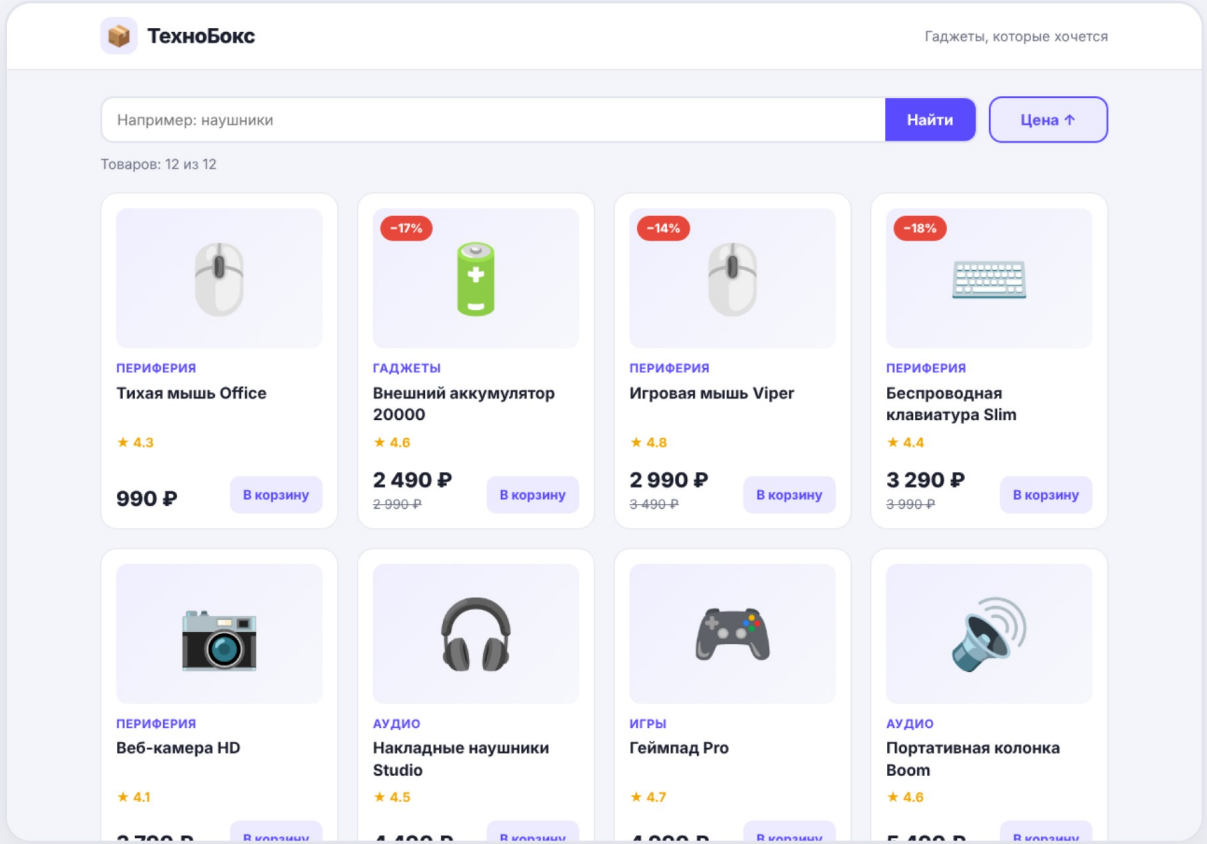
Оживляем интернет-магазин: товары из JSON, поиск и сортировка по цене

data.json | fetch | карточки | поиск | Цена ↑↓



ЧТО ПОЛУЧИТСЯ

Магазин, который умеет 4 вещи



1. **Товары из data.json**
   12 карточек появляются сами, их нет в HTML
2. **Поиск по кнопке**
   Ввёл «мышь», нажал «Найти» → только мыши
3. **Сортировка «Цена ↑↓»**
   Клик — дешёвые сверху, ещё клик — дорогие
4. **Счётчик и «пусто»**
   «Товаров: 2 из 12» и сообщение, если ничего нет

ПОДГОТОВКА · 1

Что лежит в папке и что в data.json

data.json
12 товаров. Готов

index.html
Страница и шаблон карточки. Готов

style.css
Вся красота. Готов

script.js
Пишешь ты — там подсказки TODO

ОДИН ТОВАР В data.json

```
{
  "id": 1,
  "title": "Беспроводные наушники Pulse",
  "category": "Аудио",
  "price": 7990,
  "oldPrice": 9990,
  "rating": 4.7,
  "emoji": "🎧"
}
```

Поле	Тип	Зачем
title	строка	название, по нему ищем
category	строка	подпись над названием
price	число	цена, по ней сортируем
oldPrice	число или null	старая цена (бонус)
rating	число	звёздочка ★ 4.7
emoji	строка	«картинка» товара

ПОДГОТОВКА · 2

Карта HTML: куда JS кладёт данные

index.html · главное

```
<input id="search-input">
<button id="search-btn">Найти</button>
<button id="sort-btn">Цена ↕</button>
<p id="counter">Загрузка...</p>
<section id="catalog">
  <!— ШАБЛОН карточки —>
  <article class="card">
    <div class="card__image">🎧</div>
    <p class="card__category">…
    <h3 class="card__title">…
    <p class="card__rating">…
    …цена и кнопка «В корзину»
  </article>
</section>
```

#search-input

читаем .value — что ищет пользователь

#search-btn · #sort-btn

ловим click и запускаем поиск или сортировку

#counter

пишем «Товаров: 2 из 12»

#catalog + шаблон .card

очищаем и кладём карточки **с такими же классами**, как в шаблоне — тогда CSS их сразу украсит

ПЛАН

6 шагов — после каждого проверяй страницу

1

Загрузить JSON

fetch + await → массив allProducts

✓ В консоли: «Загружено: 12»

2

Сделать карточку

createCard(товар) по шаблону
+ formatPrice

✓ В консоли видно <article class="card">

3

Нарисовать всё

render(list) + счётчик + «пусто»

✓ На странице 12 карточек

4

Мозг: state + update

состояние → список → render

✓ Всё как раньше, но через update()

5

Поиск по кнопке

click на «Найти» → filter по названию

✓ «наушники» → 2 карточки

6

Сортировка ↑↓

кнопка-переключатель по цене

✓ Первая — мышь за 990 ₽

ШАГ 1 ИЗ 6

Загружаем товары из data.json

КОД · script.js

```
let allProducts = []; // здесь будут все товары
const catalog = document.querySelector("#catalog");
const counter = document.querySelector("#counter");

async function loadProducts() {
  const response = await fetch("data.json");
  allProducts = await response.json(); // текст → массив
  console.log("Загружено:", allProducts.length);
  console.log(allProducts[0].title);
}

loadProducts(); // в самом конце файла
```

ЧТО ПРОИЗОЙДЁТ

▸ Console

```
Загружено: 12
Беспроводные наушники Pulse
```

На странице пока **ничего не меняется** — видна только карточка-пример. Данные есть, рисовать их будем на шаге 3.

⚠ Пусто и ошибка CORS?

Открой через **Live Server**, а не двойным кликом. Нет второго await → в консоли Promise вместо 12.

Как это в C# / Python: await client.GetStringAsync("data.json") + Deserialize, или json.load(open("data.json")).

ШАГ 2 ИЗ 6

Один товар → одна карточка

КОД · script.js

```
// 7990 → "7 990 ₽"
function formatPrice(price) {
  return price.toLocaleString("ru-RU") + " ₽";
}
// { … } — распаковка: достаём поля товара
function createCard({ title, category, price, rating, emoji }) {
  const card = document.createElement("article");
  card.className = "card";
  card.innerHTML = `
    <div class="card__image">${emoji}</div>
    <p class="card__category">${category}</p>
    <h3 class="card__title">${title}</h3>
    <p class="card__rating">★ ${rating}</p>
    … низ карточки — скопируй из шаблона,
    цена: ${formatPrice(price)}`;
  return card;
}
```

ЧТО ПРОИЗОЙДЁТ

Проверь в конце loadProducts:

```
console.log(createCard(allProducts[0]))
```

```
▸ <article class="card">…</article>
```

Так она будет выглядеть на шаге 3:



ШАГ 3 ИЗ 6

render(list): рисуем весь список

КОД · script.js

```
function render(list) {
  catalog.innerHTML = ""; // 1) очистить — пример исчезнет
  if (list.length === 0) { // 2а) пусто
    catalog.innerHTML = '<p class="empty">Ничего нет 😳</p>';
  } else { // 2б) каждый товар → карточка
    for (const product of list) {
      catalog.appendChild(createCard(product));
    }
  }
  const total = allProducts.length; // 3) счётчик
  counter.textContent = `Товаров: ${list.length} из ${total}`;
}
// в loadProducts вместо console.log:
render(allProducts);
```

ЧТО ПРОИЗОЙДЁТ

ДО · «Загрузка...»

пример

ПОСЛЕ · «Товаров: 12 из 12»



Пустой список — не ошибка: цикл просто не выполнится. Поэтому показываем сообщение, чтобы человек понял, что поиск сработал.

ШАГ 4 ИЗ 6

Мозг магазина: state и update()

КОД · script.js

```
// что сейчас выбрал пользователь
const state = {
  query: "", // что ищем
  sort: "none", // "none" | "asc" | "desc"
};

function update() {
  let list = allProducts; // начинаем со всех
  // шаг 5: здесь будет поиск (filter)
  // шаг 6: здесь будет сортировка (sort)
  render(list);
}
// в loadProducts: render(allProducts) → update();
```

КАК ЭТО РАБОТАЕТ

Клик «Найти» или «Цена»

↓

Меняем state

↓

update(): все товары → поиск → сортировка

↓

render(list) → страница

Пока state пустой, страница та же: 12 карточек.

Зачем это нужно: поиск и сортировка будут работать **вместе** — нашёл «мышь» и отсортировал только мышей.

ШАГ 5 ИЗ 6

Поиск по кнопке «Найти»

КОД · script.js

```
const searchInput = document.querySelector("#search-input");
const searchBtn = document.querySelector("#search-btn");

// 1) клик → запомнили слово → update
searchBtn.addEventListener("click", () ⇒ {
  state.query = searchInput.value.trim().toLowerCase();
  update();
});

// 2) в update(), перед render(list):
if (state.query ≠ "") {
  list = list.filter((p) ⇒
    p.title.toLowerCase().includes(state.query));
}
```

ЧТО ПРОИЗОЙДЁТ

␣Наушники␣ | Найти

trim убрал пробелы, **toLowerCase** — регистр: ищем **"наушники"**

«Товаров: 2 из 12»

- 🎧 Беспроводные наушники Pulse
- 🎧 Накладные наушники Studio

Пустое поле + «Найти» → снова все 12. «телевизор» → «Ничего нет 😳».

Как это в C# / Python: products.Where(p => p.Title.ToLower().Contains(q)) или [p for p in products if q in p["title"].lower()].

ШАГ 6 ИЗ 6

Кнопка «Цена ↑↓» — переключатель

1) ОБРАБОТЧИК КЛИКА

```
const sortBtn =
  document.querySelector("#sort-btn");

sortBtn.addEventListener("click", () => {
  if (state.sort === "asc") { // было ↑
    state.sort = "desc";
    sortBtn.textContent = "Цена ↓";
  } else { // было ↕ или ↓
    state.sort = "asc";
    sortBtn.textContent = "Цена ↑";
  }
  sortBtn.classList.add("active");
  update();
});
```

2) В update(), ПОСЛЕ ПОИСКА

```
if (state.sort === "asc") {
  list = [...list].sort((a, b) => a.price - b.price);
} else if (state.sort === "desc") {
  list = [...list].sort((a, b) => b.price - a.price);
}
```

ЧТО ПРОИЗОЙДЁТ

Кнопка	state	Первая карточка
Цена ↕	none	как в data.json
Цена ↑	asc	Тихая мышь · 990 ₽
Цена ↓	desc	Смарт-часы · 12 990 ₽

Искал «мышь» → ↑ → Office (990), потом Viper (2 990). Поиск **не слетает**.

БОНУСЫ · ДЛЯ ТЕХ, КТО УСПЕЛ

Делаем как в настоящем магазине

★ Поиск по Enter

```
// вынеси код поиска в функцию
function doSearch() { /* код шага 5 */ }
searchBtn.addEventListener("click", doSearch);

searchInput.addEventListener("keydown", (e) => {
  if (e.key === "Enter") doSearch();
});
```

doSearch без скобок — отдаём функцию, браузер вызовет её при клике. **keydown** — нажата клавиша, **e.key** — какая.

★★ Скидка –20%

```
// в createCard, перед return card
if (oldPrice) { // null → пропускаем
  const off = 1 - price / oldPrice;
  const badge = document.createElement("span");
  badge.className = "card__badge";
  badge.textContent = `-${Math.round(off*100)}%`;
  card.appendChild(badge);
}
```

7 990 из 9 990 → **–20%**. Так же добавь <p class="card__old-price"> — CSS её зачеркнёт.

★★★ Своя идея: вторая кнопка «Рейтинг ★» · крестик для очистки поиска · поиск ещё и по категории

ПЕРЕД СДАЧЕЙ

Проверь себя — и сравни с решением

Чек-лист

- 12 карточек, пример из HTML исчез
- Цены вида «7 990 ₽»
- «НАУШНИКИ», « наушники » → 2 товара
- Пустое поле + «Найти» → снова 12
- «Цена» переключает ↑ и ↓, кнопка подсвечена
- Поиск + сортировка работают вместе
- Нет результатов → «Ничего нет», счётчик 0
- В консоли F12 нет красного

Не работает? Смотри сюда

Виден только пример

render не вызвался или упал раньше → смотри консоль

Карточки без оформления

классы в createCard не совпадают с шаблоном

«Наушники» не находятся

toLowerCase только у одной из строк

После «Цены» поиск слетел

сортируешь allProducts, а не list внутри update

Ошибка CORS, пусто

открой через Live Server

Готово? Открой technobox-solution и сравни. Если твой код короче или иначе — это нормально, главное, чтобы чек-лист был зелёным.